

Reinforcement Learning

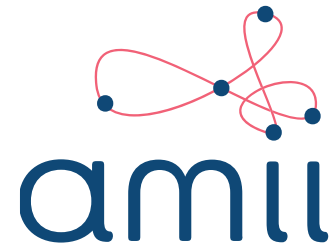
Models, Algorithms, and Applications

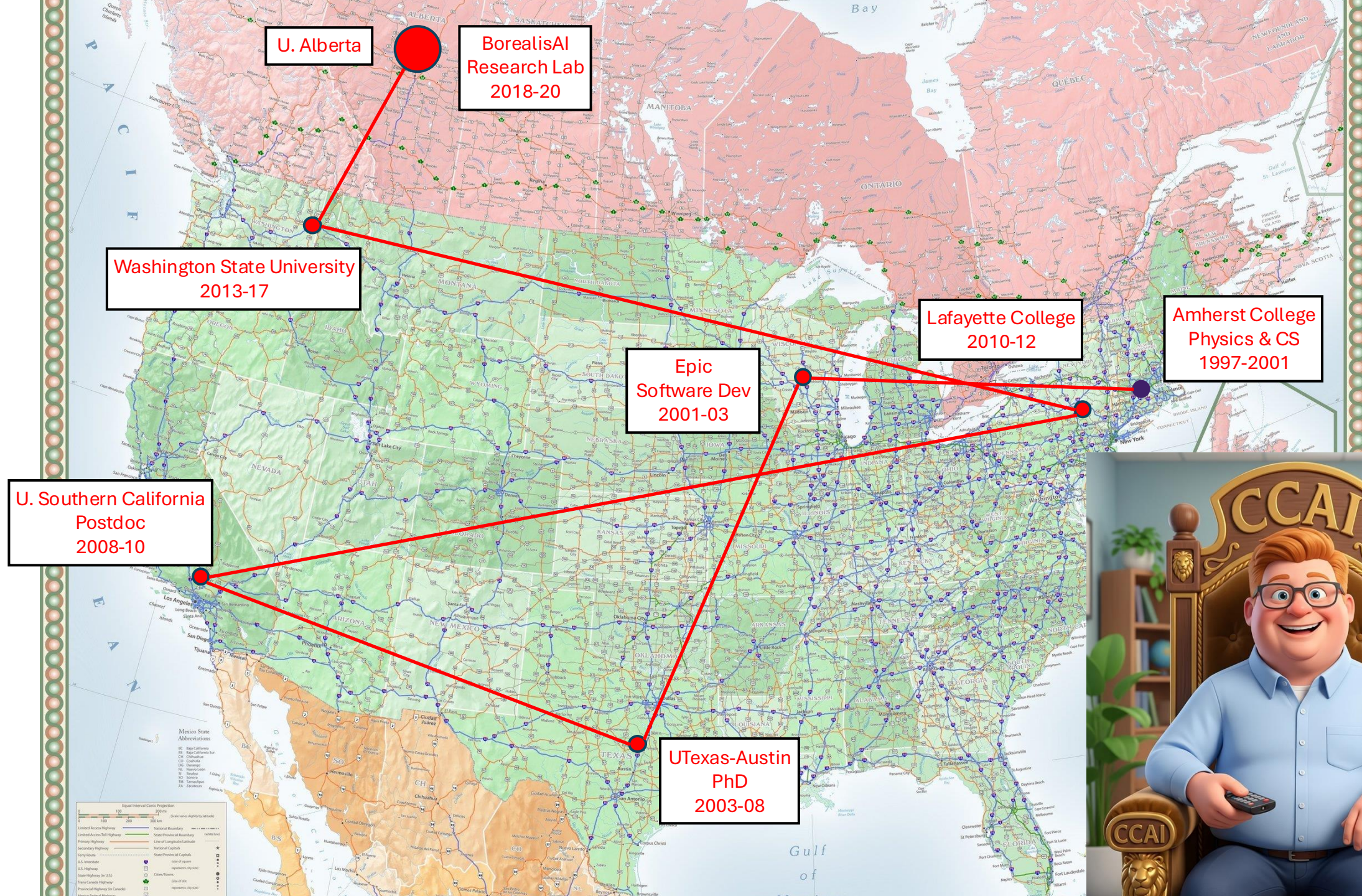
Matt Taylor

Intelligent Robot Learning Lab (<http://irll.ca>)

Amii Fellow

Canada CIFAR AI Chair, Amii





Reinforcement Learning

Models, Algorithms, and Applications

- MDPs
- Bellman equations
- Dynamic programming

Reinforcement Learning

Models, Algorithms, and Applications

- MDPs
- Bellman equations
- Dynamic programming



Reinforcement Learning

~~Models, Algorithms, and Applications~~

- MDPs
- Bellman equations
- Dynamic programming



Reinforcement Learning

~~Models, Algorithms, and Applications~~

- MDPs
- Bellman equations
- Dynamic programming

Course Roadmap

- ◆ **Part 1 — Foundations (40 min)**
 - MDPs, SSPs, and POMDPs
 - Bellman equations and policies
 - Probabilistic planning landscape
- ◆ **Part 2 — Algorithms (60 min)**
 - Dynamic Programming (VI, PI, LP/MILP)
 - Graph Search (LAO*, LDFS, Short-Sighted SSP, iDual)
 - Trial-Based (RTDP, LRTDP, RTDP-Bel)
 - Simulation-Based (UCT, PROST, POMCP, GoalHSVI)
- ◆ **Part 3 — Advanced SSP Models (20 min)**
 - Dead-ends, MAXPROB, FRET, GSSPs
 - Safety, PCTL & LTL constraints
- ◆ **Part 4 — Modeling & the IPPC (25 min)**
 - PPDDL, determinization & FF-Replan
 - RDDDL and competition highlights
- ◆ **Part 5 — Simulation-Based Planning (25 min)**
 - scikit-decision framework
 - Probabilistic planning application

Reinforcement Learning

~~Models, Algorithms, and Applications~~

- ~~• MDPs~~
- ~~• Bellman equations~~
- ~~• Dynamic programming~~

Course Roadmap

- ◆ **Part 1 — Foundations (40 min)**
 - MDPs, SSPs, and POMDPs
 - Bellman equations and policies
 - Probabilistic planning landscape
- ◆ **Part 2 — Algorithms (60 min)**
 - Dynamic Programming (VI, PI, LP/MILP)
 - Graph Search (LAO*, LDFS, Short-Sighted SSP, iDual)
 - Trial-Based (RTDP, LRTDP, RTDP-Bel)
 - Simulation-Based (UCT, PROST, POMCP, GoalHSVI)
- ◆ **Part 3 — Advanced SSP Models (20 min)**
 - Dead-ends, MAXPROB, FRET, GSSPs
 - Safety, PCTL & LTL constraints
- ◆ **Part 4 — Modeling & the IPPC (25 min)**
 - PPDDL, determinization & FF-Replan
 - RDDDL and competition highlights
- ◆ **Part 5 — Simulation-Based Planning (25 min)**
 - scikit-decision framework
 - Probabilistic planning application

Outline

- Multi-arm Bandits
- RL Successes & Settings
- Tabular example
- Q-Learning and SARSA
- Models
- Function Approximation
- DQN

- Selection of research

- Better/worse RL problems

Learning Isn't Optional

- $\mathcal{Q}$ often infinite, not "representable" (what does this mean?)
- Not every specification over a domain language corresponds to "real state"
 - ▷ Logistics: $[(\text{in } p1 \text{ airplane2}), (\text{at } p1 \text{ loc5})]$, etc
 - ▷ Blocks: $[(\text{on } A \ B), (\text{on } A \ C)], [(\text{on } A \ B), (\text{on } B \ C), (\text{on } C \ A)]$, etc
- Domain doesn't come with specification of intended states; FOL is not sufficient (e.g. Blocks)
- Known (descriptive complexity):
 - ▷ PTIME equals FOL+LFP (ordered structures)
 - ▷ NP equals $\exists\text{SO}$ (ordered structures)
 - ▷ PSPACE equals FOL+PFP (ordered structures)
- Avoid specification $\rightarrow$ synthesis not available $\rightarrow$ learning needed

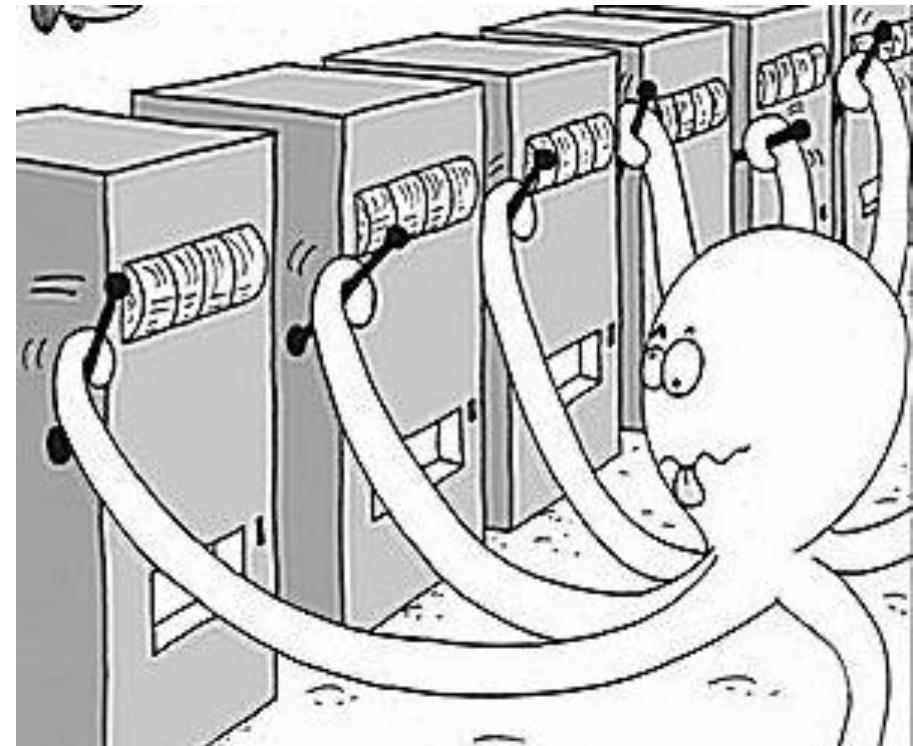
Types of Machine Learning

- Classification
- Regression
- Unsupervised Learning
- Reinforcement Learning
 - *A problem, not an approach*
 - Need to *interact*: simulator or real world

Multi-armed Bandits

- Unknown value for each arm
- Noisy payoffs

Multiple 1-armed bandits

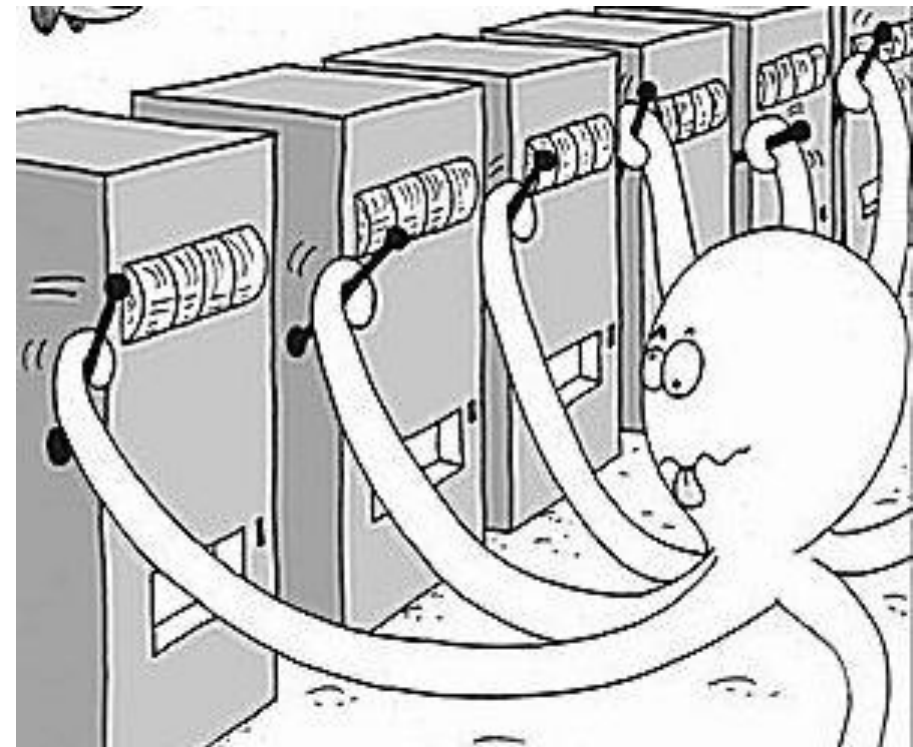


Multi-armed Bandits

- Unknown value for each arm
- Noisy payoffs

- 2, 2, 4, 4
- 1, 2, 3, 4

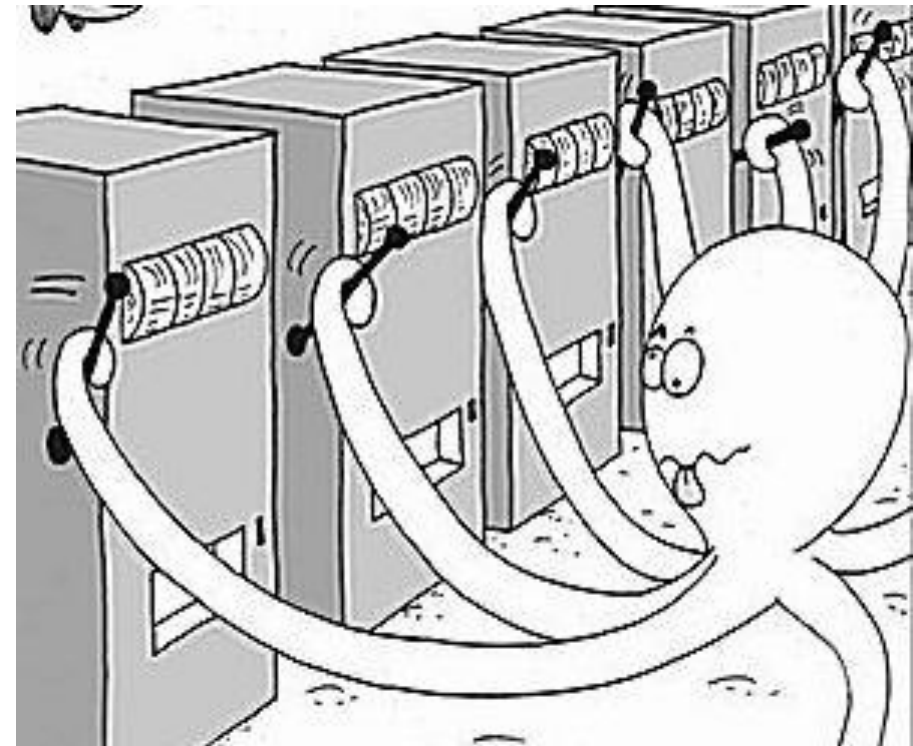
Multiple 1-armed bandits



Multi-armed Bandits

- Unknown value for each arm
- Noisy payoffs
- Estimate **value** of an arm
- Explore vs. Exploit

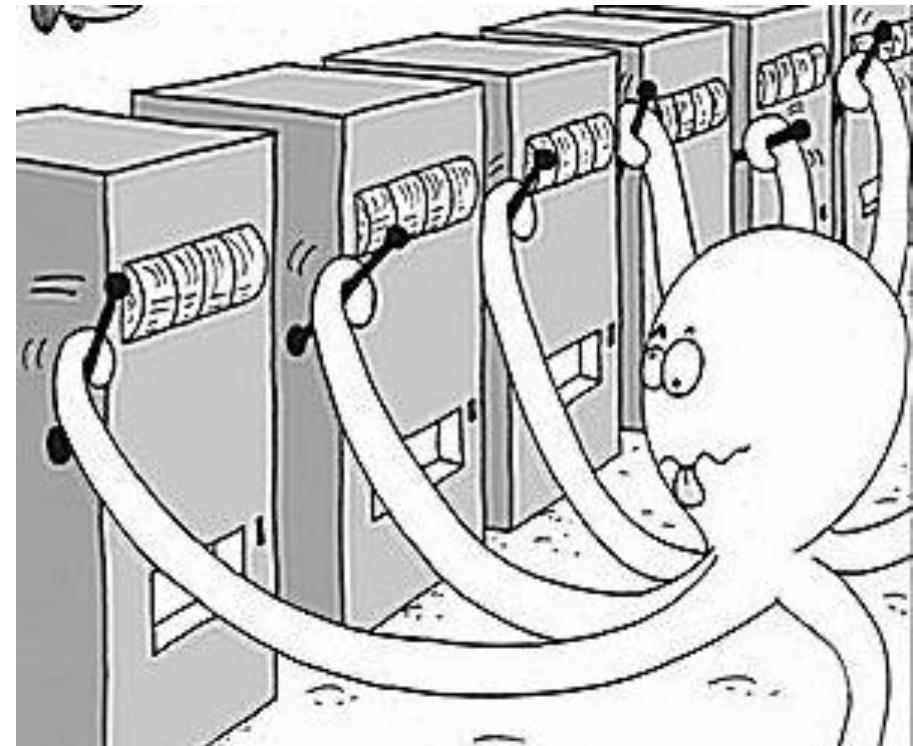
Multiple 1-armed bandits



Multi-armed Bandits

- Unknown value for each arm
 - Noisy payoffs
 - Estimate **value** of an arm
 - Explore vs. Exploit
-
- Restaurants
 - Clinical Trials
 - Ads

Multiple 1-armed bandits

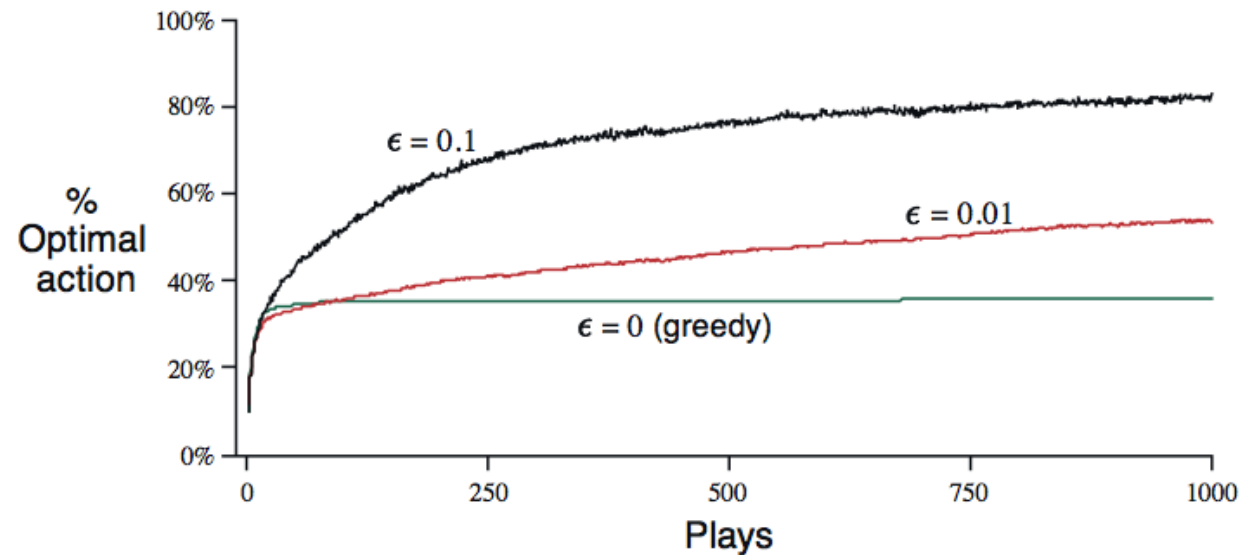
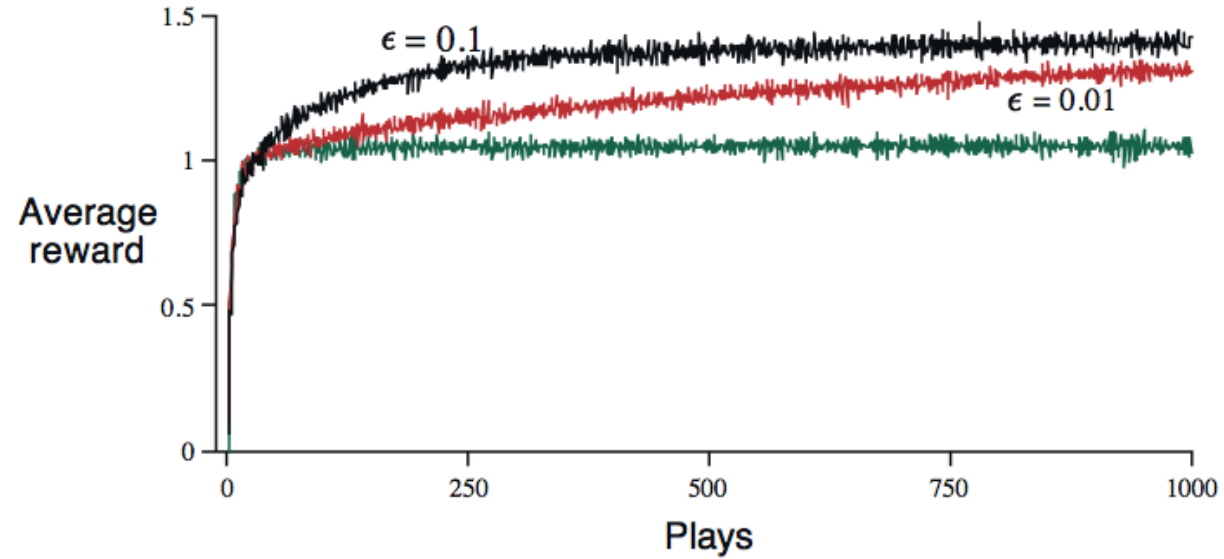


Bandit Algorithms

- Fully Greedy
- Fully Random
- ϵ -Greedy

Bandit Algorithms

- Fully Greedy
- Fully Random
- ϵ -Greedy



Bandit Algorithms

- Fully Greedy
- Fully Random
- ϵ -Greedy

$$Q_t(a) = \frac{r_1 + r_2 + \dots + r_{k_a}}{k_a}.$$

$$NewEstimate \leftarrow OldEstimate + StepSize \left[Target - OldEstimate \right].$$

Bandit Algorithms

- Fully Greedy
- Fully Random
- ϵ -Greedy
- UCB1

Bandit Algorithms

- Goal: minimize regret
- Regret: defined in terms of average reward
- Average reward of best action is μ^* and any other action j as μ_j .
There are K total actions. $T_j(n)$ is # times tried action j during our n executed actions.

$$regret(n) = \mu^* n - \sum_{j=1}^K \mathbb{E}[T_j(n)]$$

UCB1

- Calculate confidence intervals (leverage Chernoff-Hoeffding bound)
- For each action j , record average reward $\bar{x}_j$ and the number of times we've tried it as n_j . n is the total number of actions we've tried.
- Try the action that maximizes

$$\bar{x}_j + \sqrt{\frac{2 \ln n}{n_j}}$$

UCB1 regret

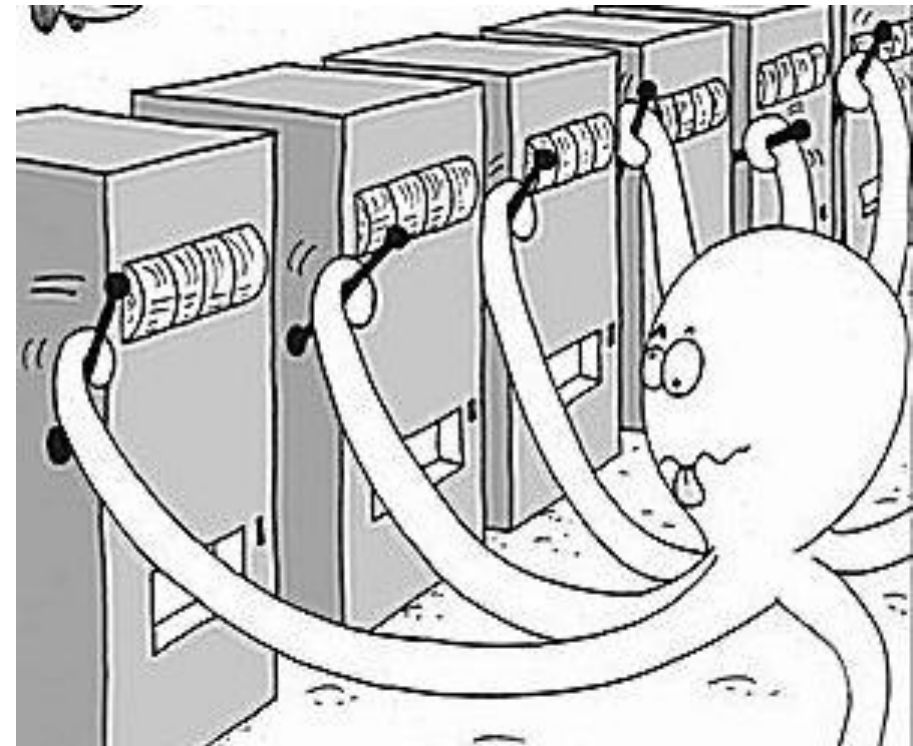
$$\sum_{j=1}^K \frac{4 \ln n}{\Delta_j} + \left(1 + \frac{\pi^2}{3}\right) \Delta_j$$

where $\Delta_j = \mu^* - \mu_j$.

Multi-armed Bandits

- Unknown value for each arm
- Noisy payoffs
- Estimate **value** of an arm
- Explore vs. Exploit
- Contextual Bandit

Multiple 1-armed bandits



Reinforcement Learning (RL) Goals

- TD-Gammon: Tesauro, 1995
- Aibo Learned Gait: Kohl & Stone, 2004
- Atari: Mnih+, 2013
- AlphaGo: Silver+, 2016

Learn autonomously, often better than hand-coded



Sony AI: Gran Turismo

Super-Human Performance in Gran Turismo Sport Using Deep Reinforcement Learning

**Florian Fuchs, Yunlong Song, Elia Kaufmann,
Davide Scaramuzza, Peter Dür**



**University of
Zurich** UZH

SONY

Sony AI: Table Tennis



Types of RL Research

- 1) Show my algo X is better than others
- 2) Understand why Y happens
- 3) Show z is possible
- 4) Understand intelligence

RL Research Settings

Given an MDP

- Learn as quickly as possible
- Get the highest performance

Given a problem

- Design an MDP and solve it
- Learn under some new assumptions

Design & solve a new problem

RL Research Settings

Given an MDP

- Learn as quickly as possible
- Get the highest performance

Atari

Robot Soccer, Go

Given a problem

- Design an MDP and solve it
- Learn under some new assumptions

TD Gammon

Gran Turismo, Table Tennis

Agar.io, Learn from a human

Design & solve a new problem

Stock trading, Chicken health

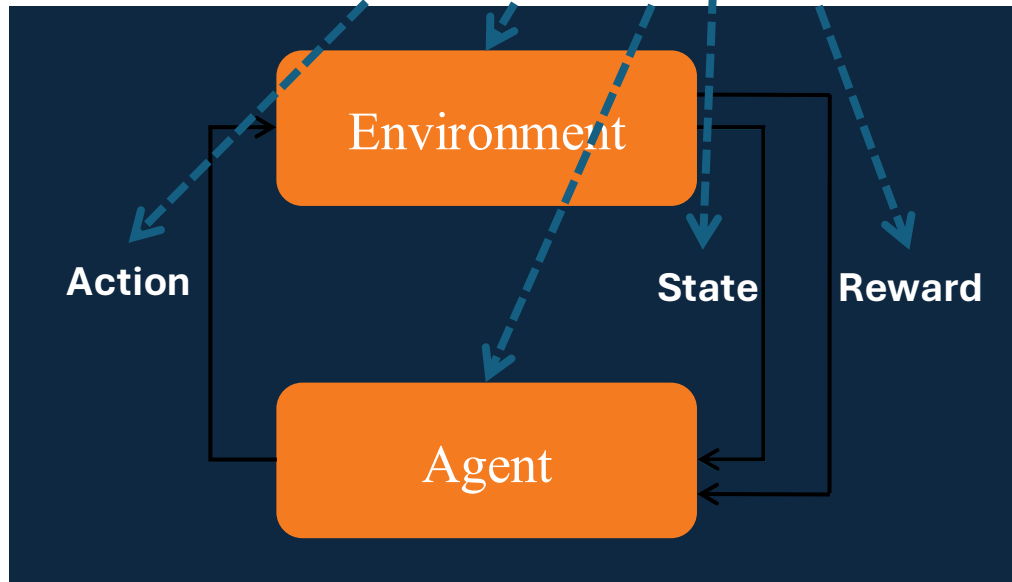
RL as a tool

- Agent can change policy
 - Reward as given
 - Representation may be given
- Pros:
 - Just specify goals
 - Can be much less work than programming
 - Super-human performance
 - Unexpected situations
- Cons:
 - Can be slow
 - Need to pick representation & rewards

Sequential Decision Making: RL

Markov Decision Process (MDP)

- S : set of states in the world
- A : set of actions an agent can perform
- $T: S \times A \rightarrow S$ (transition function)
- $R: S \rightarrow \mathcal{R}$ (environmental reward)
- $\pi: S \rightarrow A$ (policy)



RL Example

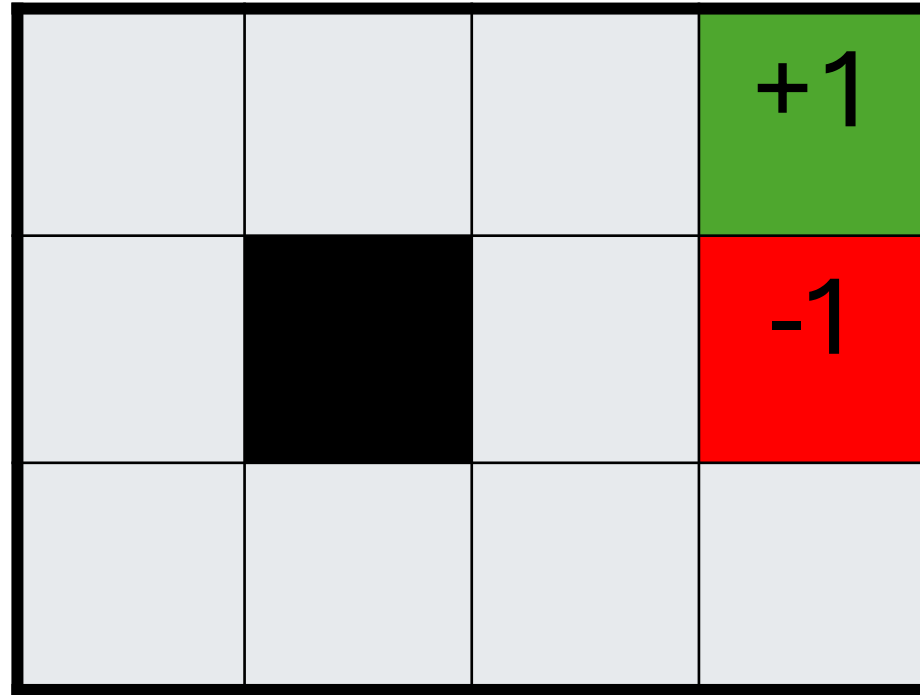
States (S): 12 in total

Actions (A): Up/Down/Left/Right

Transition Function (T): 80% works
10% goes to either side

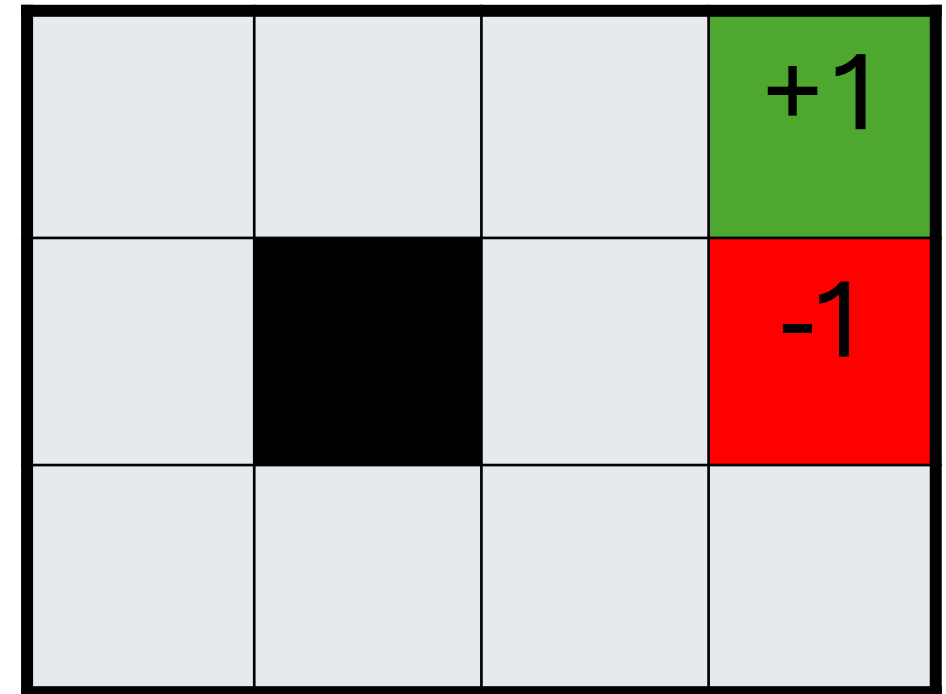
Reward Function (R): 2 end states, +/- 1

Policy π : How to act
Mapping from states to actions

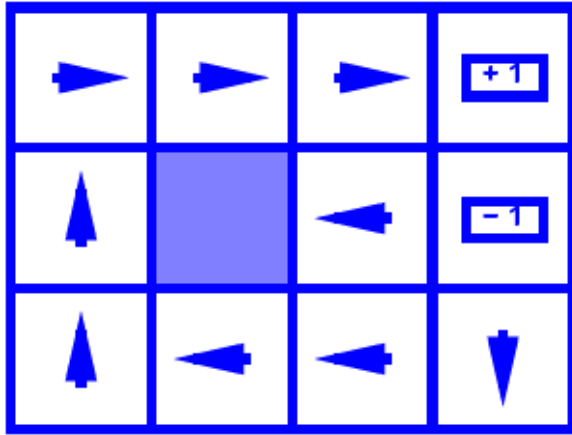


Full RL Problem

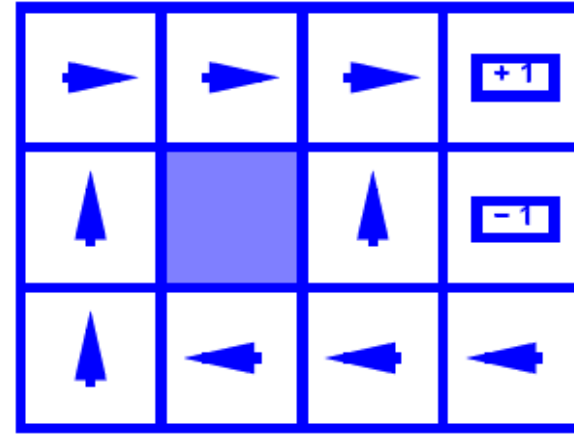
- Goal: Maximize total rewards, called the *return*
- Return = $R_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$



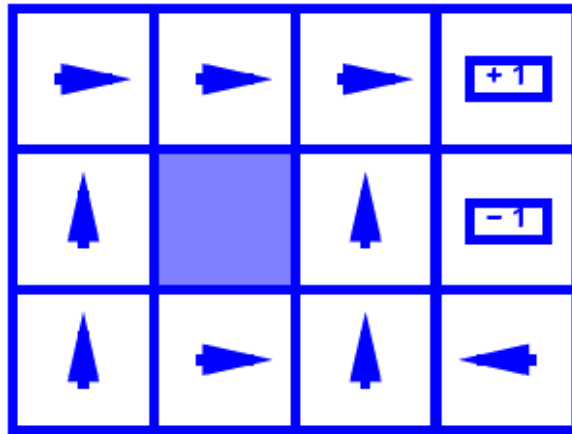
Example Optimal Policies



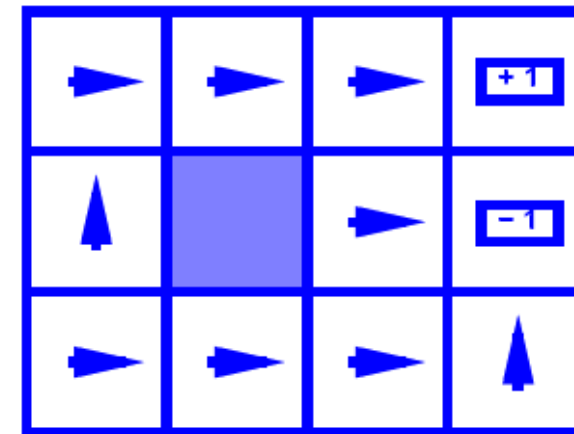
$$R(s) = -0.01$$



$$R(s) = -0.03$$



$$R(s) = -0.4$$



$$R(s) = -2.0$$

Discounting

- Discount factor: γ
- Discounted Return:

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1},$$

- t could go to infinity or γ could be 1, but not both...
- What do values of γ at 0 and 1 mean?
- Typically γ is part of the MDP

Markov Property

- One-step dynamics

$$Pr \{s_{t+1} = s', r_{t+1} = r \mid s_t, a_t, r_t, s_{t-1}, a_{t-1}, \dots, r_1, s_0, a_0\},$$

$$Pr \{s_{t+1} = s', r_{t+1} = r \mid s_t, a_t\},$$



Value Functions

- Maximize Return: $\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$

- State value function:

$$V^{\pi}(s) = E_{\pi}\{R_t | s_t = s\} = E_{\pi}\left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s \right\},$$

$$V^{\pi}(s) = \sum_a \pi(s, a) \sum_{s'} T_{ss'}^a [R_{ss'}^a + \gamma V^{\pi}(s')]$$

- If policy is deterministic:

$$V^{\pi}(s) = \sum_{s'} T_{ss'}^{\pi(s)} [R_{ss'}^{\pi(s)} + \gamma V^{\pi}(s')]$$



Bellman Equation for V^{π}

Optimal Value Functions

$$V^*(s) = \max_{\pi} V^{\pi}(s),$$

$$V^*(s) = \max_{a \in \mathcal{A}(s)} Q^{\pi^*}(s, a)$$

$$V^*(s) = \max_a \left[\sum_{s'} T_{ss'}^a [R_{ss'}^a + \gamma V^*(s')] \right]$$

Bellman Optimality Equation
for V^*

Optimal Value Functions

$$V^*(s) = \max_{\pi} V^{\pi}(s),$$

$$V^*(s) = \max_{a \in \mathcal{A}(s)} Q^{\pi^*}(s, a)$$

$$V^*(s) = \max_a \max_{s'} T_{ss'}^a [R_{ss'}^a + \gamma V^*(s')] \quad \text{Bellman Optimality Equation for } V^*$$

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a),$$

$$Q^*(s, a) = E \{ r_{t+1} + \gamma V^*(s_{t+1}) \mid s_t = s, a_t = a \}.$$

$$Q^*(s, a) = ???$$

Bellman Optimality Equation
for Q^*

Optimal Value Functions

$$V^*(s) = \max_{\pi} V^{\pi}(s),$$

$$V^*(s) = \max_{a \in \mathcal{A}(s)} Q^{\pi^*}(s, a)$$

$$V^*(s) = \max_a \square_{s'} T_{ss'}^a [R_{ss'}^a + \gamma V^*(s')]$$

Bellman Optimality Equation
for V^*

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a),$$

$$Q^*(s, a) = E \{ r_{t+1} + \gamma V^*(s_{t+1}) \mid s_t = s, a_t = a \}.$$

$$Q^*(s, a) = \square_{s'} T_{ss'}^a [R_{ss'}^a + \gamma \max_a Q^*(s', a')]$$

Bellman Optimality Equation
for Q^*

In an MDP

- Goal is to learn a policy $\pi(s) \rightarrow a$

- It should maximize the return:

- You can learn a state value function

$$V^{\pi}(s) =$$

- You could also learn an action value function

$$Q^{\pi}(s, a) =$$

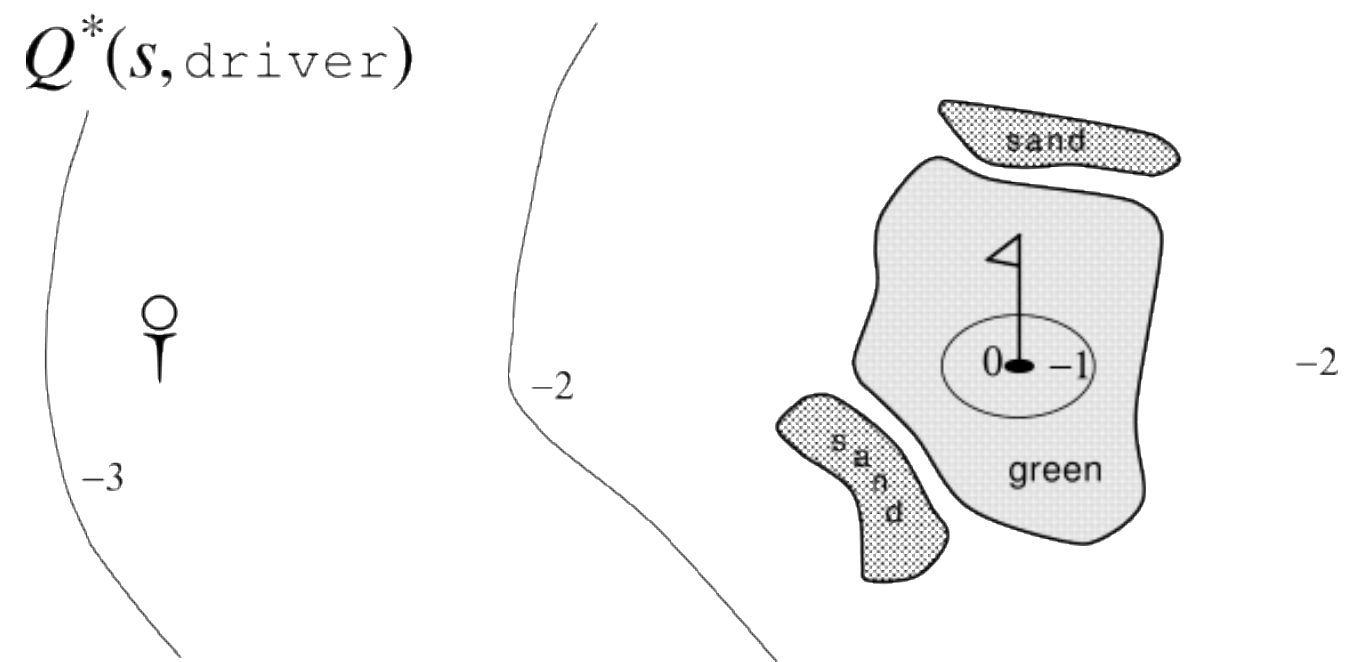
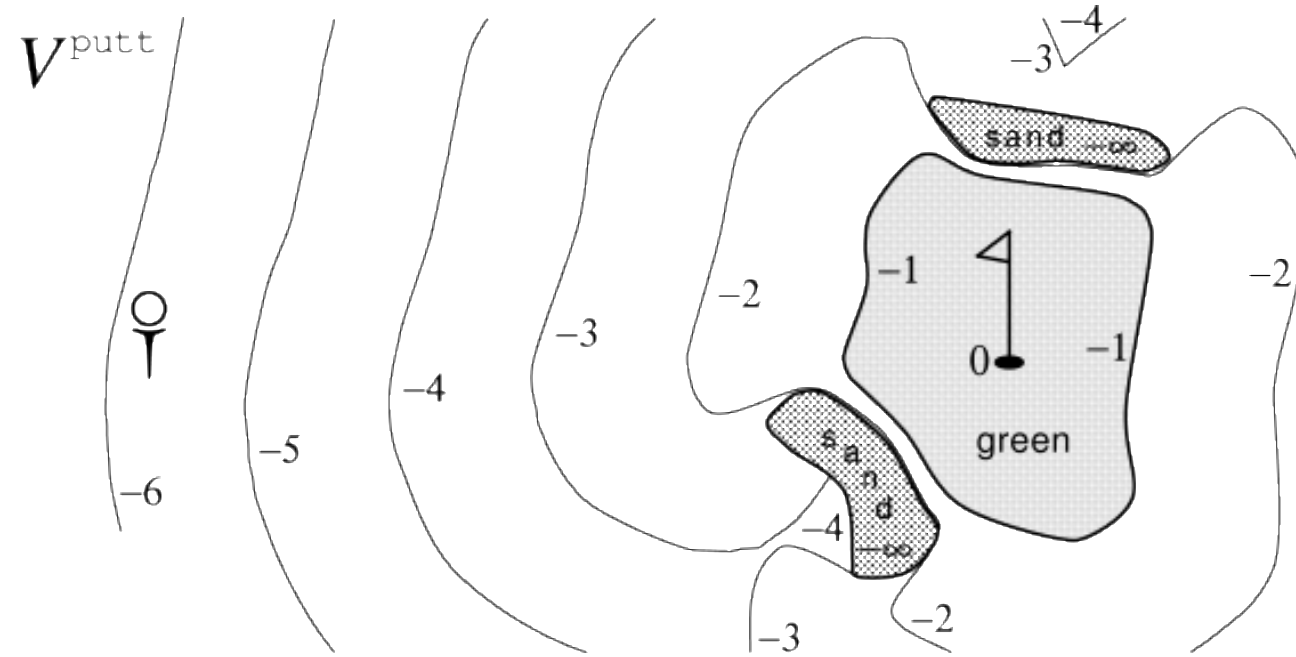
In an MDP

- Goal is to learn a policy $\pi(s) \rightarrow a$
- It should maximize the return: $\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$
- You can learn a state value function

$$V^{\pi}(s) = \mathbb{E}_{s'} T_{ss'}^{\pi(s)} [R_{ss'}^{\pi(s)} + \gamma V^{\pi}(s')]$$

- You could also learn an action value function

$$Q^{\pi}(s, a) = \mathbb{E}_{s'} T_{ss'}^{\pi(s)} [R_{ss'}^{\pi(s)} + \gamma Q^{\pi}(s', \pi(s'))]$$



- Getting V from Q

- $V(s) = \text{_____} Q(s,a)$

- Getting Q from V

- $Q(s,a) = \text{_____} + \text{_____} V(s')$

- Getting V from Q

- $V(s) = \max_a Q(s,a)$

- Getting Q from V

- $Q(s,a) = R(s,a) + \sum_{s'} T(s,a,s') V(s')$

Policy Iteration

1. Policy Evaluation

- For all states, improve estimate of $V(s)$ based on policy

2. Policy Improvement

- For all states, improve policy(s) by looking at next state values

Outline

- Multi-arm Bandits
- RL Successes & Settings
- Tabular example
- Q-Learning and SARSA
- Models
- Function Approximation
- DQN

- Selection of research

- Better/worse RL problems

TD-Learning: Prediction

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$$

Q-Learning: Control

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)].$$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Repeat (for each step of episode):

Choose A from S using policy derived from Q (e.g., ε -greedy)

Take action A , observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$$

$$S \leftarrow S';$$

until S is terminal

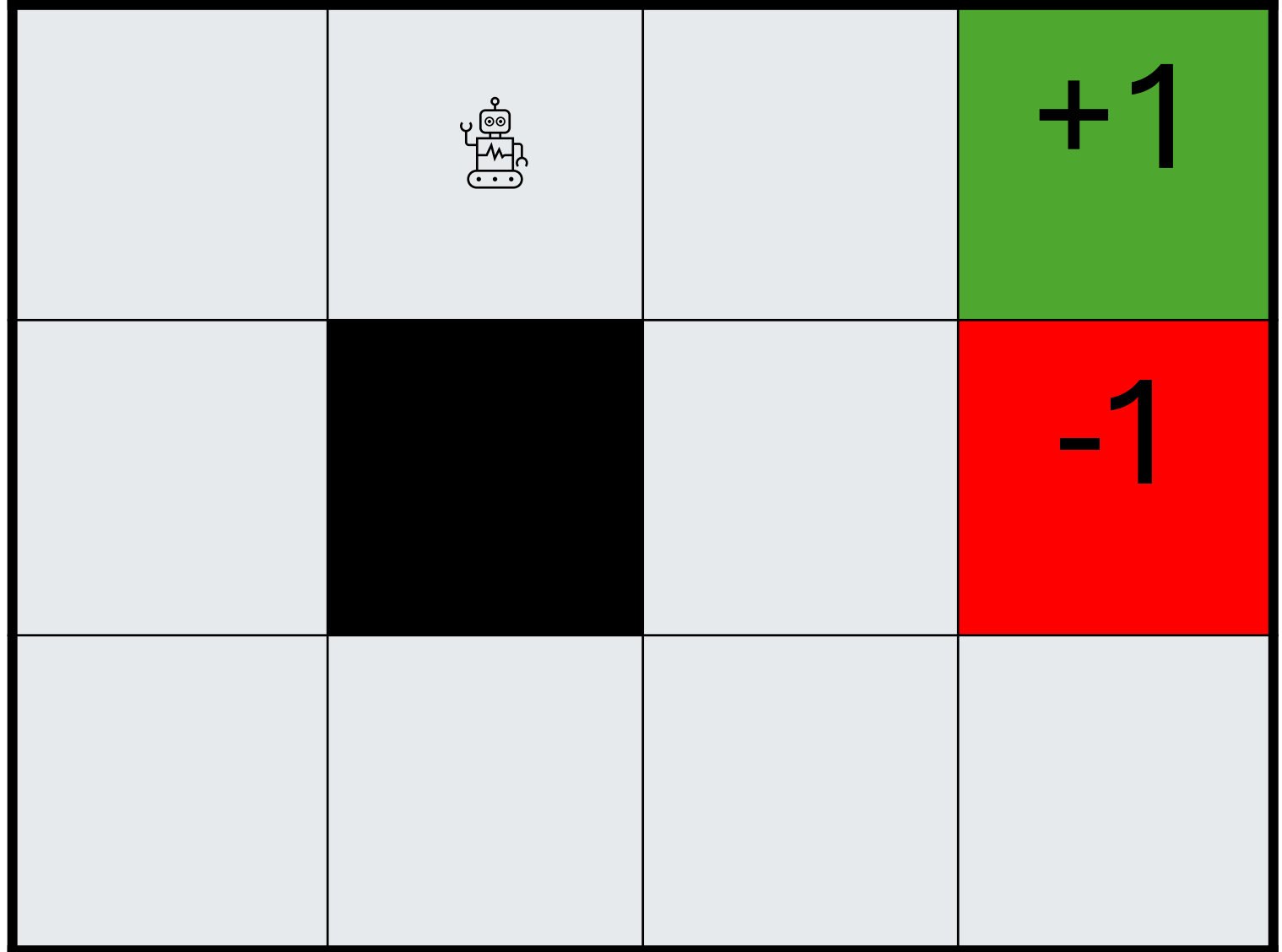
Q-Learning: Control

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right].$$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$
Repeat (for each episode):
 Initialize S
 Repeat (for each step of episode):
 Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$;
 until S is terminal

Deterministic transitions

$\alpha = 0.1, \gamma = 1.0, \text{step } r = -0.1$



Q-Learning: Control

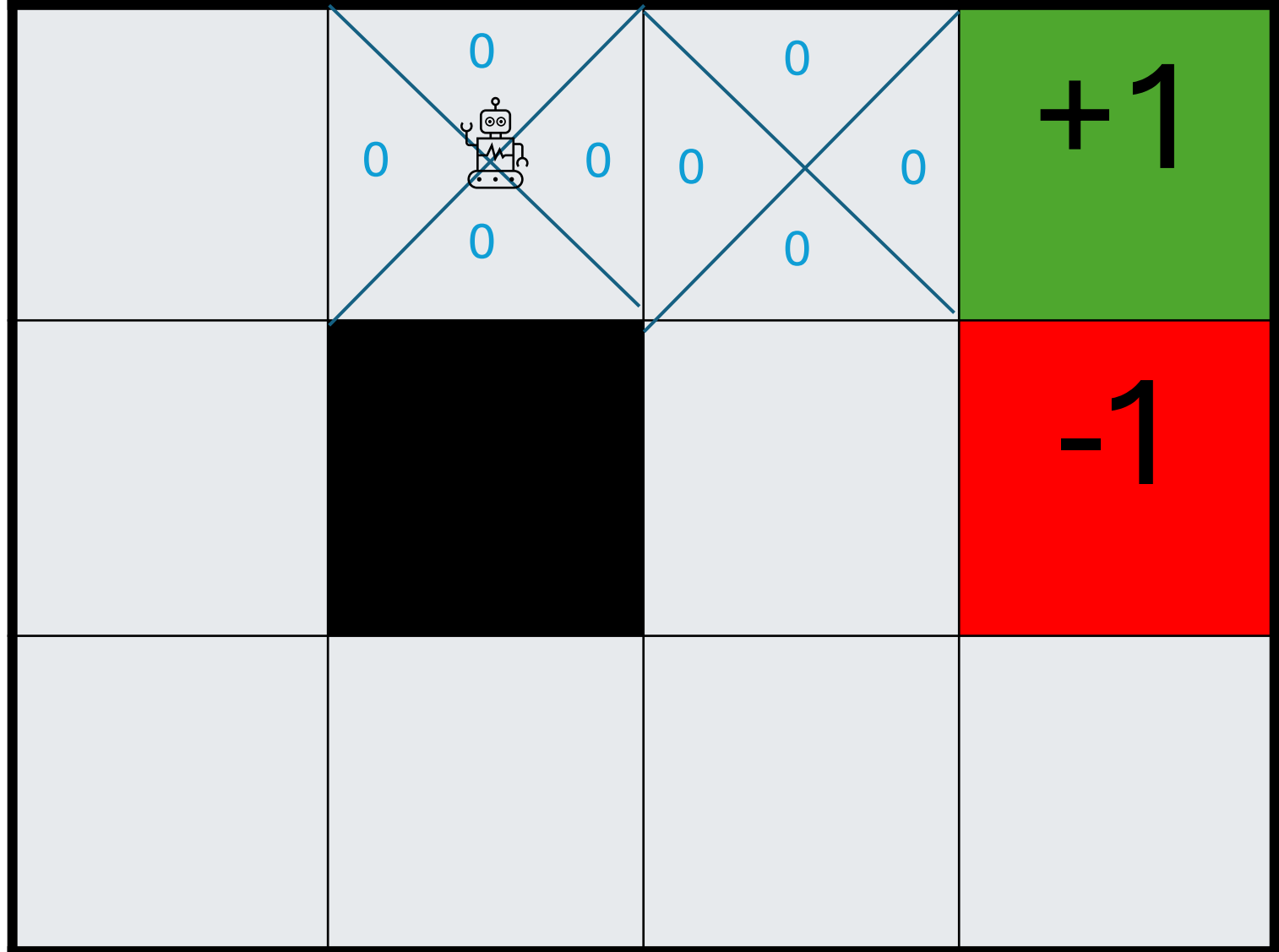
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right].$$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$
Repeat (for each episode):
 Initialize S
 Repeat (for each step of episode):
 Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$;
 until S is terminal

GO RIGHT

Deterministic transitions

$\alpha = 0.1, \gamma = 1.0, \text{step } r = -0.1$



Q-Learning: Control

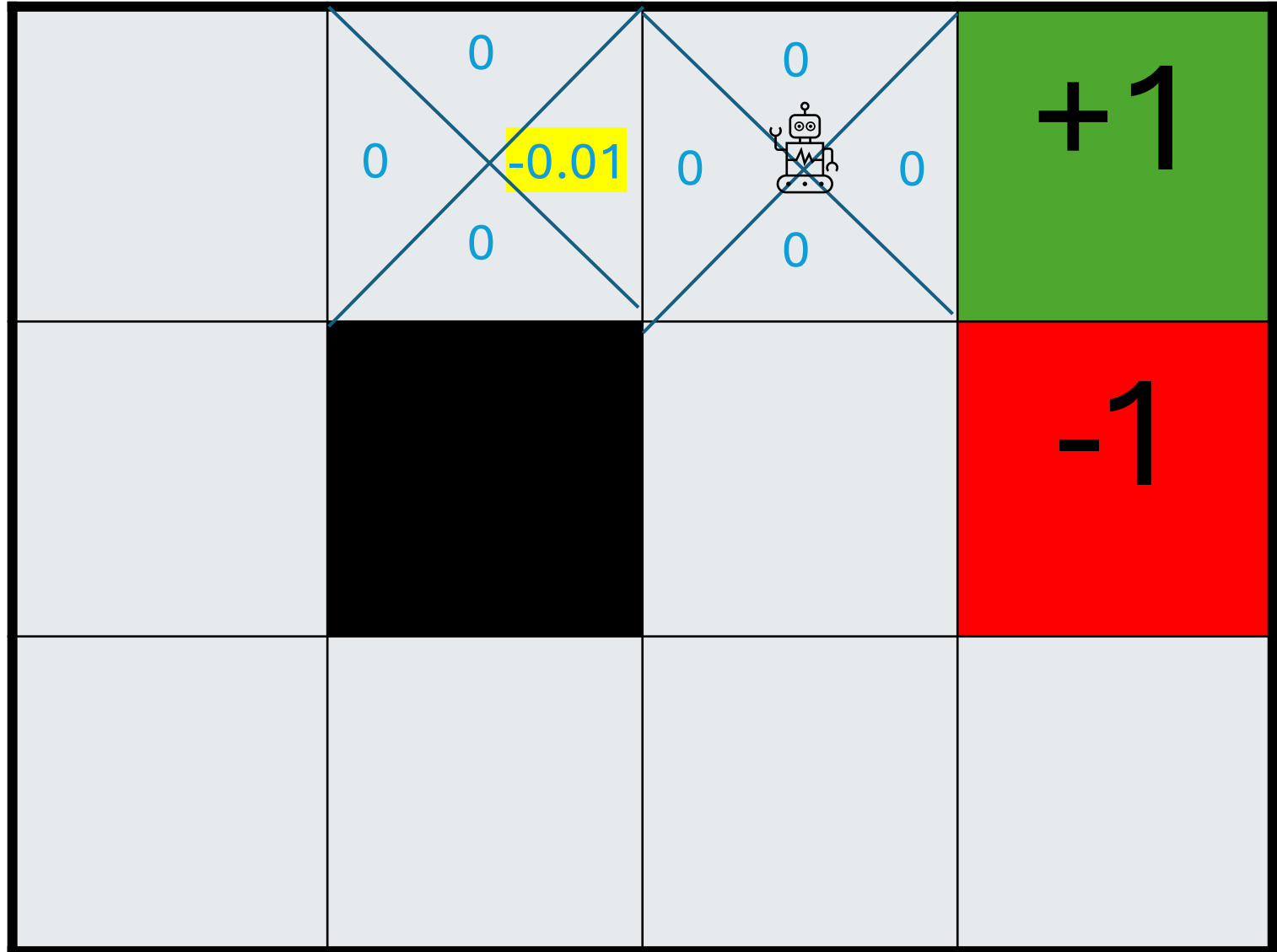
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)].$$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$
Repeat (for each episode):
 Initialize S
 Repeat (for each step of episode):
 Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$;
 until S is terminal

GO UP

Deterministic transitions

$\alpha = 0.1, \gamma = 1.0, \text{step } r = -0.1$



Q-Learning: Control

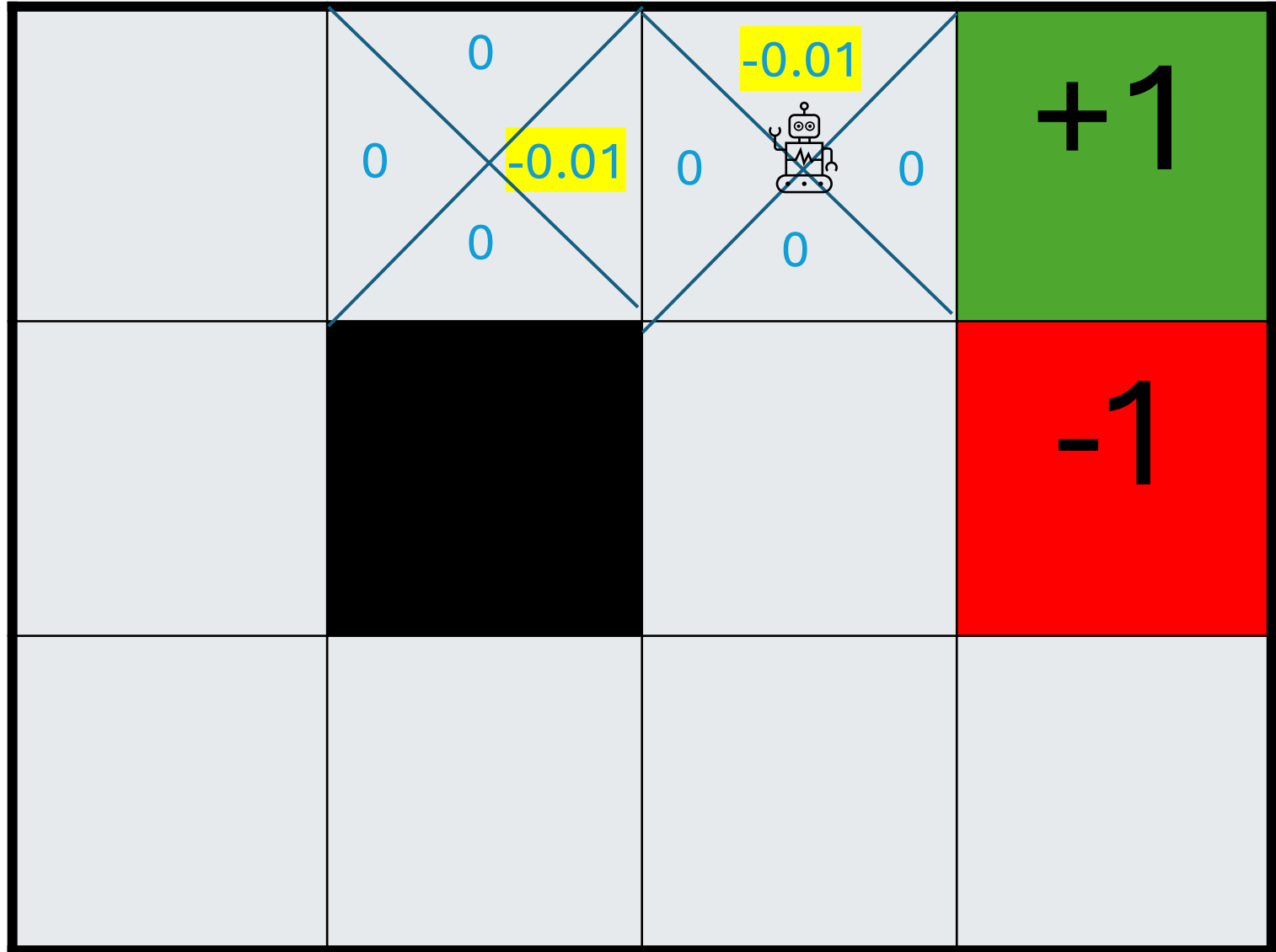
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)].$$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$
Repeat (for each episode):
 Initialize S
 Repeat (for each step of episode):
 Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$;
 until S is terminal

GO RIGHT

Deterministic transitions

$\alpha = 0.1, \gamma = 1.0, \text{step } r = -0.1$



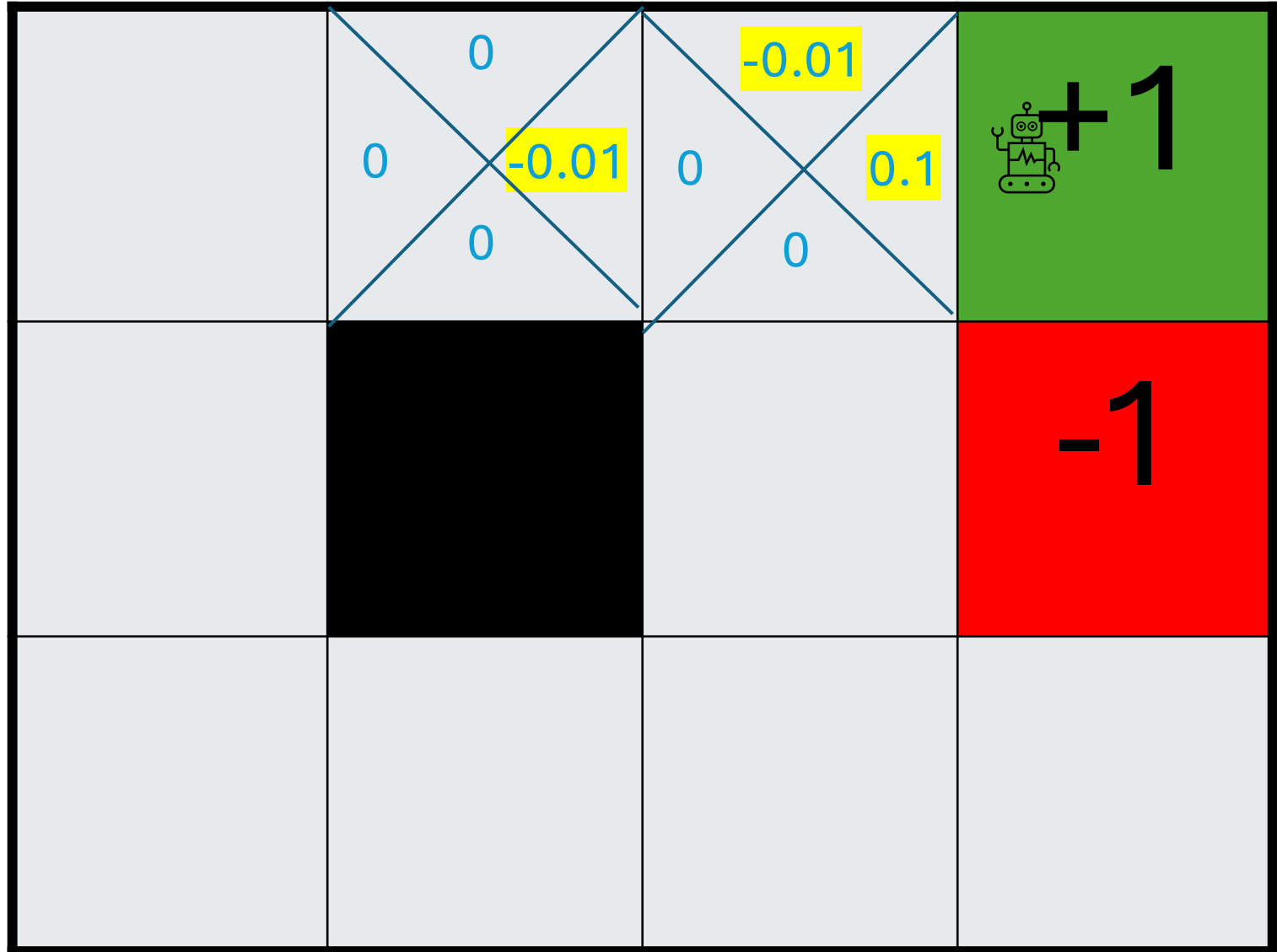
Q-Learning: Control

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right].$$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$
Repeat (for each episode):
 Initialize S
 Repeat (for each step of episode):
 Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$;
 until S is terminal

Deterministic transitions

$\alpha = 0.1, \gamma = 1.0, \text{step } r = -0.1$



Sarsa

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)].$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)].$$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Choose A from S using policy derived from Q (e.g., ε -greedy)

Repeat (for each step of episode):

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

until S is terminal

Q-Learning

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\textit{terminal-state}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Repeat (for each step of episode):

Choose A from S using policy derived from Q (e.g., ε -greedy)

Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$;

until S is terminal

SARSA

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\textit{terminal-state}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Choose A from S using policy derived from Q (e.g., ε -greedy)

Repeat (for each step of episode):

Take action A , observe R, S'

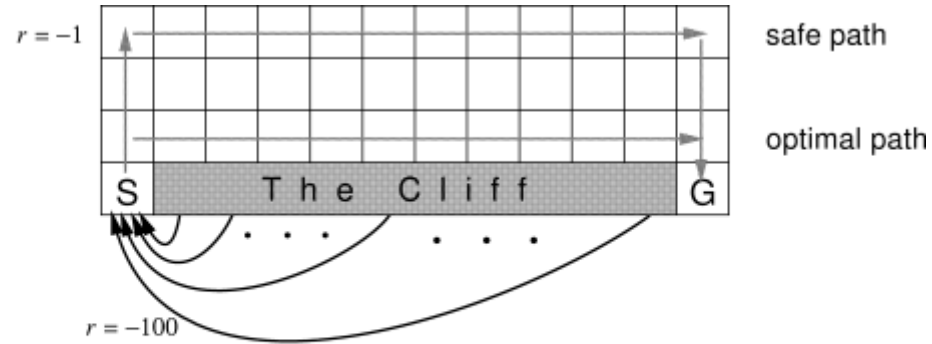
Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A'$;

until S is terminal

On-Policy Learning vs. Off-Policy Learning



ϵ -greedy, $\epsilon = 0.1$



- TD(0), prediction
- Q-Learning, Off-policy learning
- Sarsa, On-policy learning
- Sarsa & Q-learning have *no explicit policy*

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)].$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)].$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)].$$

Defining an MDP



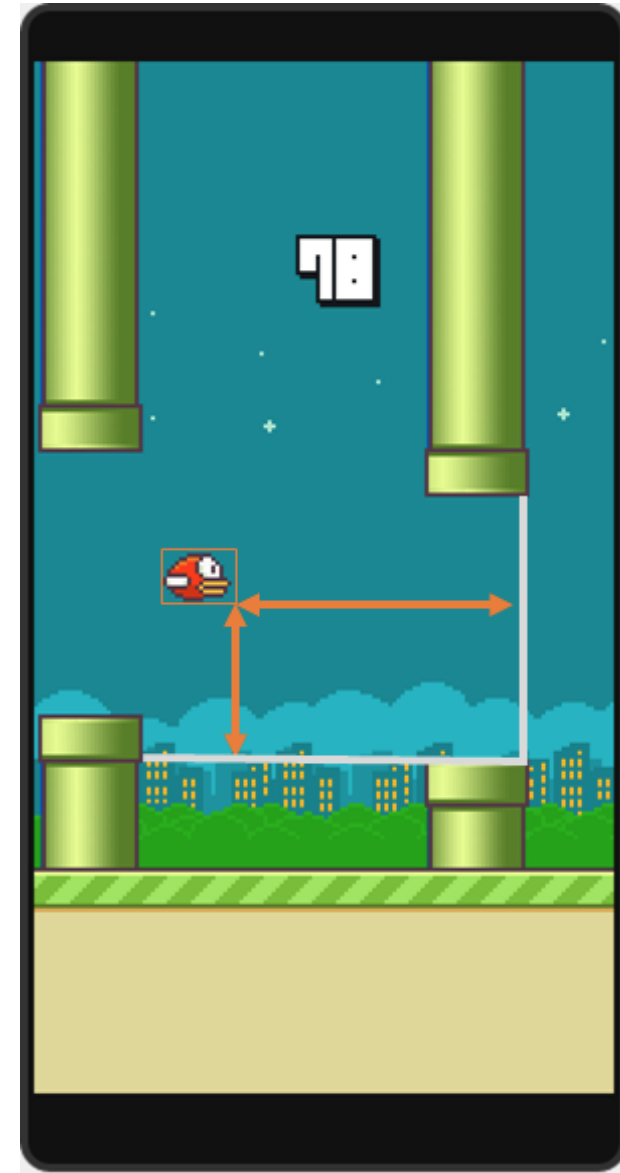
- <http://sarvagyaisha.github.io/FlappyBirdRL/>
- <https://www.youtube.com/watch?v=OJw4HTWvGdY>

Inputs

S is a set of states
 A is a set of actions
 γ the discount
 α is the step size

Local

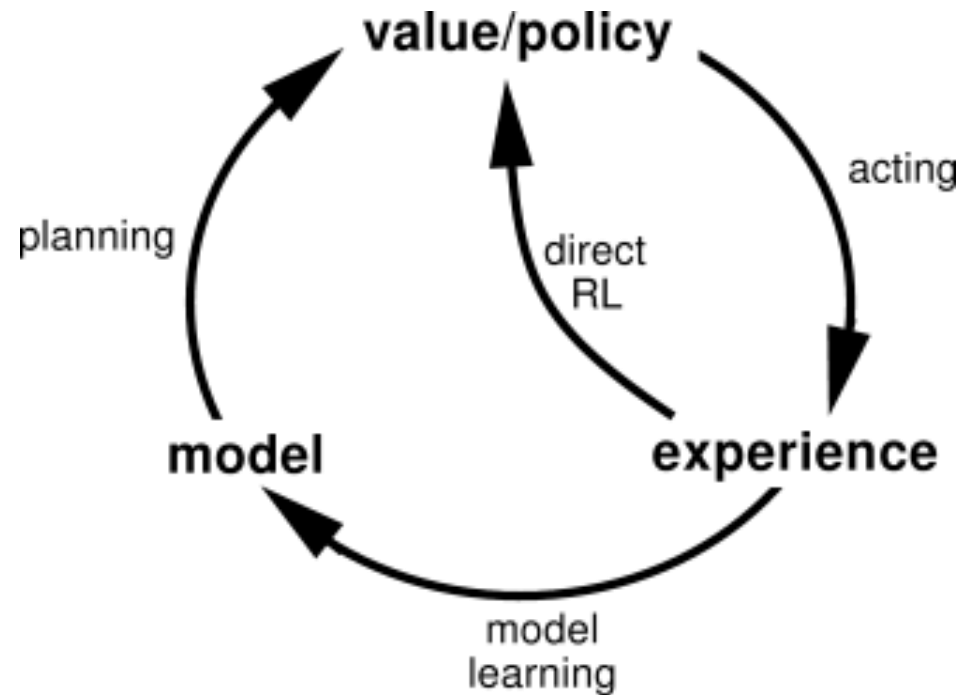
real array $Q[S,A]$
 previous state s
 previous action a
 initialize $Q[S,A]$ arbitrarily
 observe current state s
repeat
 select and carry out an action a
 observe reward r and state s'
 $Q[s,a] \leftarrow Q[s,a] + \alpha(r + \gamma \max_{a'} Q[s',a'] - Q[s,a])$
 $s \leftarrow s'$ **until** termination



Efficiency in ML / AI

1. **Data** efficiency (rate of learning)
2. **Computational** efficiency (memory, computation, communication)
3. **Researcher** efficiency (autonomy, ease of setup, parameter tuning, priors, labels, expertise)

- Learning: Uses experience
- Planning: Use *simulated* experience



Models

- A **model** lets the agent predict how the environment responds to its actions
- **Distribution model**: description of all possibilities and their probabilities
 - $p(s', r \mid s, a)$ for all s, a, s', r
- **Sample model**, aka simulation model
 - Produces sample experiences for given s, a
 - Allows reset / exploring starts
 - Often easier to obtain

Example: MCTS

- **1) Select:** Select a single node in the tree that is **not fully expanded** (i.e., at least one of its children is not yet explored)
- **2) Expand:** Expand this node by applying one available action (as defined by the MDP) from the node
- **3) Simulate:** From one of the outcomes of the expanded, perform a complete random simulation of the MDP to a terminating state (sim is finite or use estimate at some depth)
- **4) Backpropagate:** Finally, the value of the node is backpropagated to the root node, updating the value of each ancestor node on the way using expected value

Example: ICML-26

Laplacian Representations for Decision-Time Planning

Dikshant, M. Schlegel, MET, M. Machado

